

3.0 OBJECT-ORIENTED MODELING (OOM)

OOM is a way of thinking about problems using models organized around real-world concepts, where the fundamental construct is the object, which combines both data structure and behavior. The purpose of OOM is to apply object-oriented concepts to all stages of the software development life cycle. Object-oriented modeling is typically done via use cases and abstract definitions of the most important objects.

There are three types of models in object-oriented modeling and design:

- i. Class Model
- ii. State Model
- iii. Interaction Model

However, the **reasons** to model a system before writing the code are

- i. Communication,
- ii. Visualization,
- iii. Reduction of complexity
- iv. Testing a physical entity before building it.

3.1 OBJECT-ORIENTED ANALYSIS AND DESIGN

Object-oriented analysis and design (OOAD) is a software engineering methodology that involves using object-oriented concepts to design and implement software systems. It is a technical approach for analyzing and designing an application, system by applying object-oriented concepts. OOAD involves a number of techniques and practices, including object-oriented programming, design patterns, UML diagrams, and use cases.

3.2 BENEFITS OF OBJECTS-ORIENTED MODELING

The benefits of object-oriented modeling include:

- i. Faster development of software
- ii. Easier maintenance
- iii. Effective problem-solving
- iv. Better structured programs
- v. Higher code reuse

3.3 OBJECT-ORIENTED SYSTEM CONCEPTS

Object-oriented system concepts are a set of principles and practices used in software engineering to design and implement software systems. These concepts include:

- **Object:** An object is an instance of a class that encapsulates data and behavior.
- **Class:** A class is a blueprint for creating objects that defines the attributes and methods of the objects.

- **Polymorphism:** Polymorphism is the ability of objects of different classes to be treated as if they were objects of the same class.
- **Component:** A component is a modular, self-contained unit of software that can be reused in different contexts
- **Abstraction:** Abstraction is the process of identifying the essential characteristics of an object and ignoring the irrelevant details.
- **Interface:** An interface is a set of methods that define the behavior of a class.
- **Inheritance:** Inheritance is the process of creating a new class from an existing class by inheriting its attributes and methods.

3.4 STATE OF AN OBJECT

The state of an object refers to the condition of the object at a particular time period. It is a situation occurring for a finite time period in the lifetime of an object, in which it fulfills certain conditions, performs certain activities, or waits for certain events to occur.

- **Events:** Events are occurrences that may trigger a state transition in an object. Event types include an explicit signal from outside the system, an invocation from inside the system, the passage of a designated period of time, or a designated condition becoming true.
- **Transactions:** A transaction is a sequence of operations that are treated as a single unit of work.
- **Messages:** Messages are objects that describe something that has occurred in the application.

4.0 CONCEPTUAL DIAGRAMS

A conceptual diagram is a visual representation of the ways in which abstract concepts are related. It is used as an aid in visualizing processes or systems at a high level through a series of unique lines and shapes.

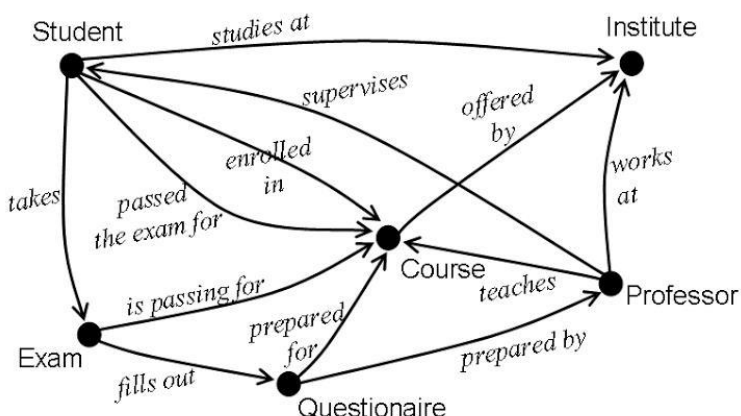


Fig 1. The conceptual diagram

There are two main types of conceptual diagrams:

- i. Causal/associative models
- ii. Descriptive/Structural diagrams

Causal/Associative Models generally, provide predictions that can be tested and falsified, whereas **descriptive/structural diagrams** provide paradigmatic/stylish ways of thinking through phenomena.

5.0 CLASS DIAGRAM

A class diagram is a type of static structure diagram in the Unified Modeling Language (UML) that describes the structure of a system by showing the system classes, their attributes, operations (or methods), and the relationships among objects.

5.1 USES OF CLASS DIAGRAM

Here are some uses of class diagrams:

- i. Class diagrams can be used to model the objects that make up the system and to display the relationships between the objects
- ii. Class diagrams describe the attributes and operations of a class
- iii. Class diagrams are used for constructing executable code of the software application
- iv. Class diagrams can be directly mapped with object-oriented languages
- v. Class diagram helps provide a clear visual representation to prevent unnecessary confusion.

5.2 PROCESS OF MODELING WITH CLASS DIAGRAM

Here's how to model with a class diagram step-by-step:

1. Identify Classes
Start by identifying the main entities in your system. These entities will become your classes. Think about the real-world objects or concepts that need to be represented in your software.
2. Define Attributes and Methods
For each class, define the attributes (properties) and methods (functions or behaviors). Attributes represent the data stored in the class, while methods define what the class can do.
3. Establish Relationships
Determine the relationships between classes. Common types of relationships include:
 - i. Association: A general connection between two classes (e.g., a customer places an order).
 - ii. Inheritance: A subclass inherits attributes and methods from a parent class (e.g., a car is a type of vehicle).
 - iii. Aggregation: A whole-part relationship where the part can exist independently of the whole (e.g., a library contains books).
 - iv. Composition: A whole-part relationship where the part cannot exist independently of the whole (e.g., a house contains rooms).
4. Draw the Diagram
Use a UML tool or diagramming software to draw your class diagram. Represent each class as a rectangle divided into three sections: the top for the class name, the middle for attributes,

and the bottom for methods. Connect classes with lines to indicate relationships and use appropriate symbols (e.g., arrows for inheritance).

5. Refine and Validate

Review your diagram to ensure it accurately represents the system. Look for missing classes, attributes, methods, or relationships. Validate the design with stakeholders and make necessary adjustments.

Example Class Diagram

Here's a simple example of a class diagram for an e-commerce system:

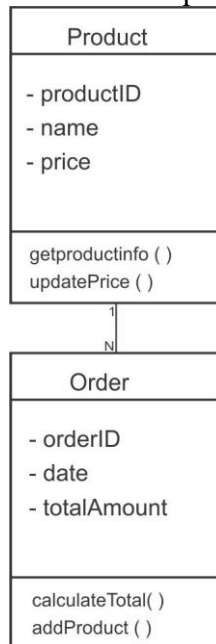


Fig 2. The Class Diagram

In this diagram:

- `Product` class has attributes `productID`, `name`, and `price`, and methods `getProductInfo` and `updatePrice`.
- `Order` class has attributes `orderID`, `date`, and `totalAmount`, and methods `calculateTotal` and `addProduct`.
- There is an association relationship between `Product` and `Order`, indicating that an order can have multiple products.

5.3 OBJECT DIAGRAM

An object diagram is a UML structural diagram that shows the instances of the classifiers in models. It provides a snapshot of the system structure, capturing the static view of the instances present and their associations.

5.4 USES OF OBJECT DIAGRAM

- i. Object diagrams show the instances of classifiers in models.

- ii. They provide a snapshot of the system representing the static view of a system.
- iii. Object diagrams are used to render a set of objects and their relationships.
- iv. Object diagrams can be useful to explain smaller portions of a system
- v. They can also be used as a quick consistency check.

5.5 PROCESS OF MODELING WITH OBJECT DIAGRAM

Here's how to model with an object diagram step-by-step:

1. **Identify Instances:** Identify the specific instances of classes that you want to represent in your diagram. These instances are real-world objects with actual data values at a particular point in time.
2. **Define Attributes:** For each object, define the attributes and their current values. Unlike class diagrams, which show potential values, object diagrams display actual data.
3. **Establish Links:** Determine the links between objects. These links represent relationships between instances, similar to the associations in class diagrams.
4. **Draw the Diagram:** Use a UML tool or diagramming software to draw your object diagram. Represent each object as a rectangle with the object name and class name, separated by a colon (e.g., `order1: Order`). Inside the rectangle, list the attribute names and their values. Connect objects with lines to indicate relationships.
5. **Validate the Diagram:** Review your diagram to ensure it accurately represents the system at that specific instance. Validate the diagram with stakeholders and make any necessary adjustments.

Example Object Diagram

Here's a simple example of an object diagram for an e-commerce system:

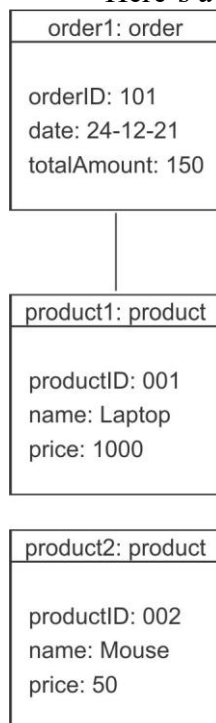


Fig 3. Object Diagram

In this diagram:

- `order1` is an instance of the `Order` class with specific attribute values.
- `product1` and `product2` are instances of the `Product` class with specific attribute values.
- There is a link between `order1` and `product1`/`product2`, indicating that these products are part of the order.

6.0 IMPLEMENTATION DIAGRAM

Implementation diagrams are used to model the physical environment of a system and how its components will be deployed.

There are two main types of implementation diagrams:

- i. Structural diagrams
- ii. Behavioral diagrams

Structural diagrams: are used to describe the static structure of a system. They include the following diagrams; Class Diagram, package diagram, object diagram, component diagram, composite structure diagram and deployment diagram

Behavioral diagrams: are used to describe the dynamic behavior of a system. They include the following diagrams; Activity diagram, use case diagram, timing diagram, state machine diagram, communication diagram and sequence diagram.